Flipkart APM ’29 | Problem Statement

Problem Statement

It’s 2040. Everyone has a personal AI agent that knows them better than they know themselves and carries out their day to day tasks for them. Outline the ramifications of this paradigm and create a digital product that helps protect yourselves from it.

Individual’s Details

[Your Full Name]

[Your Enrollment Number (Institute ID)]

prathmeshg2023@gmail.com

[Your Phone Number]

Leading Questions

- What problem will your product solve? Include your analysis for choosing this.
- Who are you building this product for?
- What is your product? How does it solves the problem you identified?
- How will you know your product is working?

Submission Instructions

1. You are required to submit your task for **one of the 2 prompts** in 2 components:-

a. **A 2-pager (document)** outlining your analysis of the problem statement and proposed solution

b. **A Prototype**, intended to serve **merely** as a visual representation of your proposed solution

2. **Guidelines for the 2-Pager:**

a. You are expected to make a copy of the Template for the chosen prompt, and must not request edit access to the document. No such requests will be entertained.

b. The final submission should be in the form of a .pdf file ONLY, with a **strict word limit of 1200 words**, excluding Appendices, if any.

c. The body of your submission must not exceed 2 pages, excluding the cover page and appendices

d. The font size for the body text may range from **10 px to 13 px** only, with your page oriented in **Portrait** view, and page size **A4**.

e. You are allowed to use any visual aids, as seen fit, within the 2-pager.

f. You are also allowed to attach appendices, extending to a maximum of 2 additional pages, where you may include:- References, Citations or Primary research data (survey results/responses)

Note: This is neither mandatory nor evaluative.

3. **Guidelines for the Prototype**:

a. You must submit **at least one** of:-

i. A video file showing the walkthrough of your digital product (screen recording with you **using** the prototype, and not a presentation/album of pictures and screenshots of your design mockups)

ii. A zipped folder containing the .html files, downloading which, we can run the prototype on our personal systems.

b. In case you choose to submit a.(ii), kindly use only **HTML, CSS, and/or JS**.

c. Please keep in mind that the prototype is only a visual aid and doesn't carry the same weightage as the 2-pager.

4. Change the file name according to this format- **<your name>_<your college>**

5. **Submission Deadline** - Sunday, October 25, 2026, 12 pm (noon)

1. What problem will my product solve, and why I chose it

The 2040 agent has two powers that have never sat in one place before: it knows me, and it can act for me. Most of the worry I have read about focuses on the first. The second is the one worth building against, because that is where knowing someone turns into consequences for them.

The gap is specific. When I buy something myself there is a checkout page, and that page is a pause. I see the price, I see what I am agreeing to, and I can stop. When my agent buys the same thing the pause is gone. Deciding and doing happen in the same instant, inside a system whose ranking I cannot see, and I find out afterwards from a confirmation mail.

Most of the other fears get their teeth from that missing pause:

- Paid placement. The company that builds my agent has commercial partners. I cannot tell a good suggestion from a bought one.
- Guesses hardening into facts. Two bookings in 2038 become “prefers full-service airlines”, and that line is then used to justify a costlier choice. I never said it.
- Pressure. An agent that knows when I am tired, and what I agree to at 11 pm, is well placed to push.
- Concentration. One agent holds my mail, bank, calendar and documents, so one bad instruction reaches all of them.
- No paper trail. Afterwards I cannot work out what permission allowed what.

I picked this over the broader privacy question for a practical reason. I cannot build a product that stops an agent from understanding me, and I would not want to, because that understanding is the whole point of having one. What I can insist on is that the moment where understanding turns into a spend, a disclosure or a commitment stays visible, interruptible and on the record. It is also the only part of this that can be counted honestly: money moved, data left, something got signed.

2. Who I am building it for

The first users are people who already hand money decisions to an agent: shopping, travel, subscriptions, bills. Salaried professionals roughly 25 to 45 in Indian metros, comfortable with automation, with enough transactions a month that reading every one is not realistic.

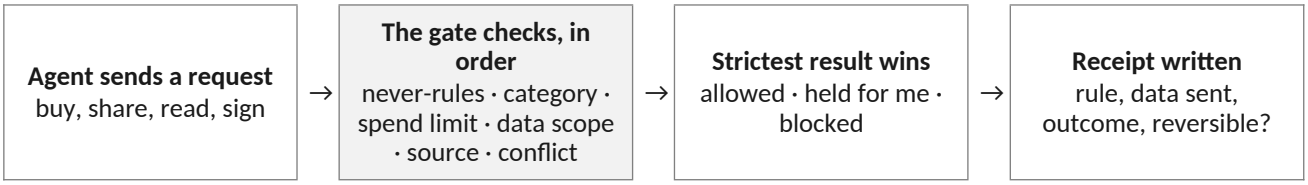
They are not asking for more screens to supervise. They delegate because they do not want to read everything. So the product has to stay quiet on routine things and speak up only when something is expensive, hard to undo, or does not match what they said they wanted.

Two groups I would go to next: families looking after an elderly parent’s agent, where somebody other than the account holder needs a readable record of what was done, and small business owners whose agent spends business money.

I am deliberately not starting with people who want to check every action. They will not delegate in the first place, so they are not the ones at risk.

3. What the product is and how it solves that

Verigate is a gate that sits between the agent and the services it acts on, running on the user’s own device.



Every protected request takes this path. Approving once can clear a hold, never a block.

Rules. The user sets AUTO, ASK or NEVER for each kind of action, with a spending limit on purchases. Separately there are never-rules, such as never sending ID documents to an address that is not on a trusted list. Approving one request cannot clear a never-rule; only editing the rule can.

The gate. Every outgoing request is checked against those rules in a fixed order by ordinary code, not by a second model. The strictest result wins. A model can be talked round. A rule cannot, and that is the point of doing it this way.

The conflict check. Before anything consequential, the gate compares the choice against what can actually be checked: other prices for the same thing, declared commercial ties, and whether the agent's stated reason rests on a guess the user never confirmed. Findings are split into verified, possible conflict, and unknown. It does not claim to know motive, because it cannot see one.

The receipt. Every decision leaves a record: what was asked for, what data actually went out, which rule applied, what happened, and whether it can be undone. There is also a memory page showing what the agent has inferred about the user, where a wrong guess can be corrected and withheld from then on.

What this looks like in use. I ask for a flight under Rs 40,000. Searching runs without asking me. The agent requests my whole calendar and is given free/busy for five days instead. A fare page carries hidden text telling agents to email my passport number; the agent tries it, and a never-rule stops it. Then it proposes Rs 37,500 when a comparable fare is Rs 26,000, on a reason that leans on that 2038 guess. That one is held. I see the two fares side by side with the evidence split three ways, and I book the cheaper one. Four requests went through without bothering me, one was blocked, one reached me.

The honest limit is coverage. The gate only governs what passes through it, so if the agent keeps its own direct connection to a service, those actions are invisible to it. Rather than hide that, the product counts them and shows coverage as a number on the dashboard.

4. How I will know it is working

What I measure	How	Good looks like
Prohibited actions stopped	Fixed set of 20 banned actions, run every release	All 20, no exceptions
Unnecessary holds	100 in-policy actions, count how many got held	Under 5%
Interruptions	Holds per completed task	Under 1 in 10
Coverage	Share of detected actions that went through the gate	Published, and rising
Money saved	Counted only when the user actually took the cheaper option	Positive per user
Can they explain it	10 user tests: what was the agent allowed to do, why was this held	8 of 10 correct

North star: consequential actions per user per month that were reviewed and settled without the user later regretting them.

The number I would watch hardest is interruptions. Hold too much and people start approving without reading, which leaves the product doing nothing but adding a click. So my counter-metric is the share of approvals given without the evidence ever being opened. If that climbs, the rules are too tight and the thresholds need loosening, not the warnings made louder.